



Fakultät Informatik

From CAD Software to Slicer

Workflow Optimization using Print Hints in 3MF Files for 3D Printing

Bachelorarbeit im Studiengang Medieninformatik

vorgelegt von

Luise Hartdegen

Matrikelnummer 364 2168

Erstgutachter: Prof. Dr. Ulrich von Zadow

Zweitgutachter: Prof. Dr. Timo Götzelmann

© 2025

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Abstract

With the increasing accessibility of consumer-grade 3D printers, an ever-growing number of users are engaging in designing and fabricating their own models. Despite this trend, the transition from digital model to printed object still involves several steps that require technical knowledge and repetitive manual configuration. In particular, defining and reapplying material and printing parameters across multiple design iterations often results in redundant effort and increased potential for error.

To address this issue, the presented concept introduces the idea of ‘print hints’ embedded directly within the 3D model to retain information about relevant printing parameters. These hints enable a more seamless workflow by linking model components to specific presets and allowing for automatic or semi-automatic reassignment of parameters during the import into slicer software. This approach reduces cognitive load, minimises repetitive tasks, and supports a more efficient and user-friendly iterative design process, while preserving user control and flexibility.

Kurzdarstellung

Mit der zunehmenden Verfügbarkeit von 3D-Druckern für Endverbraucher beschäftigt sich auch eine stetig wachsende Zahl von Nutzer*innen mit dem Entwurf und der Herstellung eigener Modelle. Trotz dieses Trends umfasst der Prozess vom digitalen Modell zum gedruckten Objekt weiterhin mehrere Schritte, die hohes technisches Wissen und wiederholte manuelle Konfiguration erfordern. Insbesondere das Definieren und Anwenden von Material- und Druckparametern über mehrere Designiterationen hinweg führt häufig zu redundantem Aufwand und einer erhöhten Fehleranfälligkeit.

Um dieses Problem zu adressieren, führt das vorgestellte Konzept sogenannte “Print Hints” ein, die direkt im 3D-Modell eingebettet werden, um eine Zuordnung zu relevanten Druckparametern zu speichern. Diese Hinweise ermöglichen einen nahtloseren Arbeitsablauf, indem sie Modellkomponenten mit spezifischen Voreinstellungen verknüpfen und eine automatische oder halbautomatische Zuweisung von Parameter-Voreinstellungen beim Import in die Slicer-Software erlauben. Der Ansatz reduziert die kognitive Belastung, minimiert wiederkehrende Aufgaben und unterstützt einen effizienteren und benutzerfreundlicheren iterativen Designprozess, während gleichzeitig die Kontrolle und Flexibilität der Nutzer*innen erhalten bleibt.

Contents

1. Introduction	1
2. Background	5
2.1. State-of-the-art Workflow	5
2.2. 3D Modeling Software	7
2.3. Slicer	8
2.3.1. Filament Presets	8
2.3.2. Printing Parameters	9
2.3.3. Modifiers	9
2.3.4. Slicer Files	10
2.4. File Formats used in 3D Printing Context	10
2.4.1. STL	10
2.4.2. OBJ	11
2.4.3. 3MF	11
2.5. Software-specific Workflow	12
2.5.1. <i>Autodesk Fusion</i>	12
2.5.2. <i>OrcaSlicer</i>	13
3. Concept	15
3.1. Design Requirements	15
3.2. Concept Development	16
3.3. File Format Considerations	16
3.4. User Interaction	17
4. Technical Implementation	18
4.1. Technical Analysis of the 3MF File Structure	18
4.1.1. <i>Fusion</i> 's Export Behaviour	19
4.2. Analysis <i>OrcaSlicer</i> 's 3MF Import Logic	19
4.2.1. Internal Data Handling	20
4.3. User Interface Adaption	20
5. Results	22
5.1. Planned Work	22

6. Conclusion	24
A. Supplemental Information	25
List of Figures	29
List of Listings	30
Bibliography	31

Chapter 1.

Introduction

It is one of the clearest paradoxes of modern work that computer-based systems designed to reduce task complexity and cognitive workload actually often impose even greater demands and stresses on the very individuals they are supposed to be helping.[1, p. 222]

Recent years have seen a rising popularity of digital fabrication within the hobbyist or so-called maker community, as well as in educational and professional settings. This growth can be attributed to the increasing availability of devices such as 3D printers and laser cutters in public spaces, along with the declining cost of these devices for end-users.[2]

Similar to traditional printers, 3D printers serve as a bridge between the digital and the physical, transforming virtual designs into tangible objects. Unlike conventional printing, which produces two-dimensional images or text on paper, additive manufacturing (AM) creates three-dimensional artifacts by depositing material layer by layer.[3] The applications of this technology are diverse, ranging from toys to household items, from replacements components to artistic creations, from educational models to prototypes for mass manufacturing, and ultimately to finished products in the emerging market of customised mass production, exemplified by websites such as *Thingiverse.com*, where users can upload their 3D models.

Initially, when the technology became widely available, the user base consisted primarily of individuals who were not only interested in the final result, but also demonstrated a keen interest in the technology itself and the process of creating models and setting up the printer. It is from this community that the so-called *makerspaces* emerged. These are spaces that provide affordable access to digital fabrication tools and act as knowledge hubs, often with financial support from their users.[4]

As usability and automation have advanced, 3D printers¹ have become increasingly accessible to casual users. This has facilitated their integration into educational contexts,

¹The term “3D printers” used in this paper specifically refers to smaller desktop machines that use Fused Filament Fabrication (FFF) / Fused Deposition Modeling (FDM) technology

where they serve as tools for teaching skills such as design, engineering, and project management. As Walter-Herrmann and Büching note, digital fabrication “accelerates the processes of ideation and invention. It eliminates manual dexterity as the ‘middle-man’ in transforming an idea into a product, so students can focus their attention on improving the design rather than taking care of mundane issues with the materials”[5, p. 209]. Consequently, 3D printing enables faster iteration and experimentation in design-oriented learning environments.

Nowadays, 3D printers are frequently available in schools, libraries, and community spaces, where staff may not possess the same level of technical expertise as those in dedicated makerspaces. This shift has led to a more heterogeneous user base, differing in technical knowledge, learning intent, and focus on the underlying technology versus the final product. Despite these differences, the general workflow from idea to finished object follows a consistent pattern.

Using the example of a simple box with a snap-fit closure and multiple material colours (1.1), this process typically looks as follows: after coming up with an idea, a user first develops a digital 3D model using computer-aided design (CAD) software. They then assign materials, adjust printing parameters, and apply modifiers to influence specific properties — such as increasing wall thickness for greater structural integrity — before converting the model into printer-specific instructions. Following

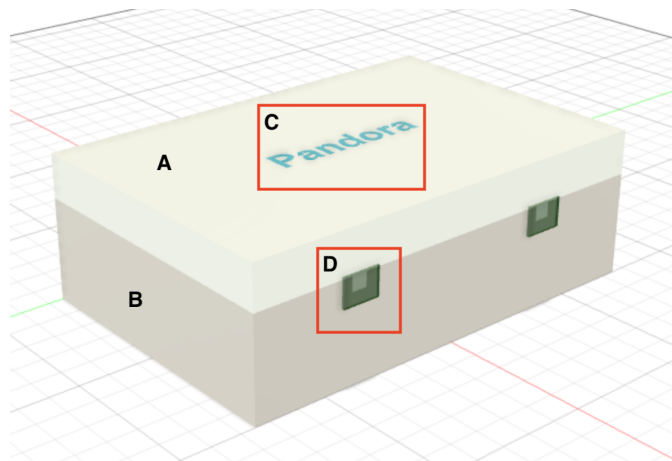


Figure 1.1.: 3D model of snap-fit closure box as example. A: Top. B: Bottom. C: Integrated text to be printed in different colour.

D: Closure with surrounding modifier.

the physical setup of the printer, the object is produced layer by layer. The resulting artefact is subsequently evaluated and compared with the intended outcome, informing adjustments either to the model or the printing parameters for subsequent iterations[6, p. 52]. This iterative process often involves repetitive steps that require users to manually document or recall detailed settings, introducing inefficiencies and potential for error.

Due to the highly specific use cases and differing requirements, a “standard 3D printing workflow [...] requires disparate pieces of software, which then leave behind only a fragmented record of the fabrication process. CAD models are stored and shared as static

geometry files. Separately, operators can tweak relevant settings in CAM software called a slicer, which can then be saved as a slicing profile.”[7]

The expanding and diversifying user base has prompted discussions within the academic human-computer interaction (HCI) community about how to streamline the process, lower hurdles and simplify the sometimes rather complex steps involved. Even though the technology itself improved in regards to usability and maintainability, the skills required to operate a 3D printer remain high.[8]

Various approaches have been proposed to address this challenge, ranging from simplified 3D modelling software[9], and alternative methods of process documentation[7] to the use of conversational agents as human-machine interfaces.[8] While these efforts contribute valuable insights, they typically focus on isolated segments of the fabrication workflow rather than on the seamless integration of multiple applications into a continuous process. Accordingly, there remains considerable potential within the field of HCI to enhance existing tools by considering the interdependencies among different stages of the workflow.

One particularly relevant aspect concerns the management and documentation of printing parameters. Accurate documentation is essential for both reproducing successful results and diagnosing issues when they occur. To ensure consistency across iterations, printing parameters should remain unchanged unless they have been explicitly identified as a source of error. This requirement, however, necessitates that users either remember their settings or record them externally, often outside the software ecosystem in use. Although many slicer applications provide functionality to store user-defined presets, this feature becomes insufficient as project complexity increases, since users still need to track which presets were applied to which parts of a model. In this regard, the software currently exhibits a lack of granularity.

Focusing on the relationship between individual parts of the model and their associated printing parameters, this work examines the feasibility of implementing ‘print hints’ early in the design process to strengthen the integration between 3D modelling and slicing software, thereby enhancing the overall user experience. Print hints are markers added to the 3D model during the modelling phase that the slicer can process semi-automatically. The underlying concept is to allow users to associate these markers with their own material or printing presets while simultaneously storing the resulting mappings when the model is imported into the slicer. When the same marker is encountered in a subsequent iteration or within a new model, the system can automatically reapply the corresponding preset while still providing the user with the option to modify it.

The overarching goal is to streamline the iterative design workflow by reducing the need to repeatedly configure identical printing parameters in each iteration, therefore lowering

users' cognitive workload and potential for error. The prototype developed in this study primarily explores workflow improvements using existing software functionalities, complemented by a newly designed user interface and modified handling procedure during the model import stage in the slicer.

Chapter 2.

Background

Since the proposed solution builds upon existing features within modelling and slicing software, it is essential to establish a fundamental understanding of these systems and the file formats connecting them. The next chapter therefore outlines the key concepts and technologies underlying 3D modelling and slicing, providing the basis for the subsequent analysis and prototype development.

2.1. State-of-the-art Workflow

The following section examines the general workflow described in the previous chapter in greater detail, further using the 3D model illustrated in Figure 1.1 as an example.

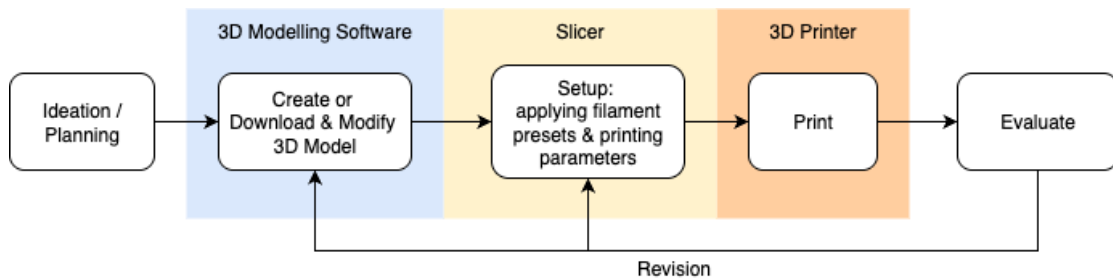


Figure 2.1.: Iterative workflow in 3D printing highlighting the step to be optimised

1. 3D Model

After coming up with an idea, the user either creates the 3D model from scratch or downloads one from the various databases available online and optionally modifies it. Creating and modifying a model requires highly domain-specific knowledge and skills. Although a wide range of educational resources — including videos, written tutorials, and documentation — are readily available for all levels of expertise, newcomers often require substantial time, practice, and effort before their designs approach the intended outcome.

2. Print Setup

As the next step, the user has to choose the printing material, called filament, for different parts, e. g. printing the box (1.1, A and B) itself in white PLA and the text (C) in a different color PLA, and set at the very least the temperature according to the materials specifications in case they differ from the presets used. Oftentimes further parameters or presets are needed to determine factors like supports¹, infill², printing speed and heating bed temperature.

In this case, the green boxes around the closures (D) need to be marked as modifiers to increase the wall loops in the main model for added stability. Furthermore the box is designed to be printed as two separate bodies, one each for the top and bottom, which means the top including all its text bodies have to be placed on the printing bed accordingly to save on material and avoid support structures.

3. Slicing

The model must then be translated into layer-by-layer instructions for the printer, including the generation of supports and necessary helping structures such as the wiping tower for multi-material printing. Although this is a technically complex process, the complexity is handled entirely by the software and therefore does not present significant challenges for the user.

4. Printing

After physically setting up the printer, i. e. loading the chosen materials and optionally performing calibrations, the model can be printed using the slicers instructions.

5. Evaluation

Especially for newcomers and casual makers, the initial print often does not meet expectations. The printed object must therefore be evaluated for potential design flaws or printing errors. This often leads to revisions to either the 3D model or the printing setup, initiating a new iteration of the workflow.

Ergo, two parts of the process require a considerable specialised knowledge: the creation of the 3D model and the preparation of the model for printing. Particularly given the iterative nature of the workflow, several repetitive steps can be identified.

¹3D printers build a model by printing one layer on top of another. While overhangs can be printed up to a certain degree and distance, in some cases a supporting structure is needed as basis for upper layers.[10, p. 152]

²Infill is the structure printed within a hollow 3D model. It can vary regarding the amount, usually indicated in percent, as well as the pattern. Both of these affect the model's resulting stability.[10, p. 155]

For instance, when multiple materials are used — such as printing text in a different colour — the user has to assign the corresponding material presets to the respective model components in each iteration. As project complexity increases, this task can become increasingly tedious and prone to error.

A similar challenge arises when model parts are employed as modifiers, for example to strengthen specific areas by increasing wall thickness around a snap-fit closure. In such cases, the modifiers must also be redefined with every iteration, and unlike material assignments, they cannot always be saved as presets. This further reinforces the need for systematic documentation and reapplication of settings throughout the process.

2.2. 3D Modeling Software

When realising a design from scratch - rather than downloading and modifying an existing model — the user typically employs 3D modelling software. Such software enables the transformation of an idea or a two-dimensional sketch into a three-dimensional solid or surface representation, thereby fully defining the external geometry of the object.[10, p. 4]

A fundamental distinction can be drawn between software that focuses primarily on surface representation and that which supports solid modelling. For additive manufacturing, a fully enclosed model, i.e. one that encloses a complete volume, is essential. This requirement is often difficult to fulfil when using surface-modelling software, as surfaces are described by boundary curves that can be challenging to join with the precision and continuity necessary to form a closed body. In contrast, software developed for manufacturing-oriented modelling is designed to maintain geometric and topological consistency, typically ensuring that resulting models are fully enclosed and possess a defined volume.[10, p. 21] Although such software frequently includes additional functionalities, such as computer-aided manufacturing and physical simulations, only the computer-aided design aspect is relevant to the workflow considered in this study.

Solid-modelling systems used in manufacturing contexts are based on precise geometric definitions. Typically, the process begins with two-dimensional sketches that incorporate parametric constraints specifying dimensions, angles, and positional relationships among components. These sketches can be positioned on any plane within the coordinate system or on the surface of existing geometry. Subsequent operations, such as extrusion, cutting, or lofting, then transform these sketches into three-dimensional volumes.

Modern CAD systems commonly rely on boundary representations that utilise Non-Uniform Rational B-Splines (NURBS) to mathematically define surfaces and curves.[10,

p. 21] To facilitate downstream applications such as slicing, these systems also provide tools for exporting models into a wide range of common interchange file formats.

2.3. Slicer

The fundamental principle of 3D printing is that components are created through the additive process of layering material, with each layer representing a thin cross-section of the part derived from the original CAD data.[10, p. 2] The task of dividing a model into these discrete layers and generating sequential printing instructions corresponds to the second and third steps of the aforementioned workflow, and constitutes the primary function of the software tool commonly referred to as a slicer. Although slicer applications may include limited features for creating or editing 3D models, their focus lies in preparing a completed model for printing and translating the three-dimensional data into printer-specific instructions.[4]

Slicers are capable of importing 3D models in a range of file formats and transforming them to the software’s internal representation. Within the slicer interface, the model’s individual components are placed on a virtual depiction of the printer’s build platform or *print bed*, the physical surface on which the object will be produced. Since printers have varying build volumes, the print bed measurements as well as the available height need to be available for correct part placement and to optimise print efficiency and material usage. While users may configure these parameters manually, most slicers provide a pre-defined library of printer profiles, often including models from multiple manufacturers, to facilitate setup.

In extrusion-based 3D printing, material characteristics such as rheology (the flow behaviour of molten materials) or melting temperature are as relevant as machine configurations, including print speed, nozzle geometry, and heating profiles.[7] These variables introduce a considerable degree of complexity to the printing process, as each parameter can influence the final print quality in different ways. For the purpose of conceptual clarity, this paper will differentiate between *filament* and *printing parameters*, even though this distinction is not necessarily reflected in the slicer software’s user interface.

2.3.1. Filament Presets

3D printers support a wide variety of filament types, each with distinct material properties and potentially manufacturer-specific characteristics. As a result, the associated printing parameters can vary considerably between materials. To reduce this complexity, slicer software typically provides a selection of predefined filament presets, which may

be either generic or tailored to specific manufacturers. A preset is a collection of parameters that can be applied collectively to streamline the configuration process, ensuring that all model components, that the corresponding filament is assigned to, are treated uniformly. These parameters generally include the filament's name and base material, as well as recommended settings such as nozzle temperatures, extrusion speed, and cooling behaviour. Users may duplicate and modify these presets to fine-tune individual parameters according to their specific printer setup or desired print quality, thereby enabling the reuse of optimised configurations across multiple projects.

2.3.2. Printing Parameters

Additional parameters frequently adjusted during print preparation include those related to print quality (e.g. layer height, line width, seam positioning, precision, and bridging behaviour) as well as parameters that influence the model's mechanical strength (e.g. number of wall loops, shell thickness, and infill density or pattern). Further settings determine the type and placement of support structures, including threshold angles at which supports are generated. Unlike filament presets, these parameters are not necessarily tied to the selected material and may vary between individual model components. Slicer software typically provides a number of standard presets for such configurations as well, while also allowing users to modify or create custom profiles as needed.

2.3.3. Modifiers

As previously stated in the introductory example, specific components within a 3D model may act as a modifier. They typically change printing parameters, such as the number of wall loops or infill density, locally for a limited region of the model. For example, additional reinforcements may be applied to the area around the snap-fit closure, which is subjected to repeated mechanical stress, by increasing the number of wall loops.

Although it is possible to add such objects within the slicer's user interface, this approach tends to be cumbersome and imprecise. Therefore users may prefer to define modifying objects during the 3D modelling stage, thus avoiding repetitive manual positioning in the slicer during each design iteration. It is important to note that the term 'modifier' is used in this paper as an umbrella term encompassing not only conventional modifiers but also support blockers and enforcers.

2.3.4. Slicer Files

Once the model is sufficiently configured, the slicer converts it into a series of discrete layers, described as paths for the printer to follow. Due to the nature of this process, slicers typically manage two distinct file types. Before slicing, the 3D model is stored as a project file including geometric data as well as the associated parameters, allowing for the configuration to be reopened, modified and reused at a later point, independent of a specific printer. For this purpose, slicers generally use either the open-source format 3MF or a proprietary file format. Following the slicing process, the resulting model is exported in a format specific to a single printer ecosystem. This format is commonly referred to as G-code.^[11]

A variety of slicers are available nowadays, both general ones and versions customised by printer manufacturers. Since getting all the settings right for a particular 3D printer can be complex, some manufacturers prefer to adapt one of the open-source slicers, creating a version for their machines.^[6, p. 58] Thus, the majority of commonly used slicers share the same code base, derived from an ancestor called *Slic3r*. The common lineage and the need to keep these projects open-source³, has created a collaborative environment in which new features introduced in one version are oftentimes ported to others as well.

2.4. File Formats used in 3D Printing Context

A plethora of alternative file formats are available to represent 3D models in general, however not all of these are equally appropriate for digital fabrication and only a select few have been adopted by the AM community. Hence, the following paragraphs will delve into the three most prevalent formats utilised in 3D printing.

2.4.1. STL

The STL (“STereoLithography”) format was introduced in the 1980s and has since gained widespread support among software applications used in digital fabrication. It is considered to be the most prominent and prevalent mesh file format, despite its stringent limitations. The contents can be either encoded in binary or in ASCII text format. The latter is less common, but it is used for illustration and educational purposes. It describes an approximation of the surface geometry in the form of a mesh of triangles, each identified by a unit normal vector as well as three vertices, as seen in [A.1](#). The normal vector is equal to the length “1”, however it does not specify whether the dimensions

³*Slic3r* is licensed under the *GNU Affero General Public License*, version 3, which requires derivative works to remain open-source.

are millimetres, inches or another unit entirely. Consequently, while the proportions of a model remain constant, the overall size depends on the user’s knowledge about the model. Due to its simplicity, the file does not contain information regarding color, text or other common CAD model attributes.[10, p. 341 ff.][11]

Furthermore, since the specification requires each triangle to share two of its vertices with the triangle adjacent to it, thereby creating one continuous solid model, technically there will be exactly one body in the resulting file. For a model comprised of multiple parts, the user therefore is required to choose whether to combine all bodies into one or export each body as a separate file. However, modern slicer software is capable of separating parts into disparate bodies, even if the user chooses to export a multi-body model into one file.

2.4.2. OBJ

The OBJ file format is considered to be the second most common file format as it contains color and texture information in addition to the options STL presents. It was developed by Wavefront Technologies and later introduced to various other 3D graphics systems due to its simplicity and open-source license. Like STL files, it supports both ASCII text as well as binary encoding. While the 3D model does approximate the surface of the objects similar to STL files, OBJ also supports free-form surfaces and curves.[11]

As can be seen in A.2, the model is represented by vertices as well. Each group of vertices is prefaced by a body name and optionally a material, referencing a separate `.mtl` (material template library) file.

Common problems with this format include the file’s complexity, making it difficult to repair non-closed surfaces, as well as the need for additional plug-ins in some CAD systems to export it. On top of that, having more than one file to describe a single 3D model is a potential cause of issues, especially when sharing, importing and transferring files.

2.4.3. 3MF

3MF (“3D manufacturing format”) has been developed by the 3MF Consortium, founded by Microsoft, Autodesk, HP, 3D Systems, Prusa Research and several other key players in the field, to create a 3D printing file format that addresses the shortcomings of previous standards. In particular the possibility for extensions and the support of colors as well as several different 3D model representations are aspects in which the format is specifically designed to support interoperability between CAD and AM systems.[11]

The two formats described above can generally be opened by a standard text editor to get a human readable result. 3MF on the other hand is “a ZIP archive that contains a complex file structure, defining a 3D model in Extensible Markup Language (XML).”[12] Same as the other two formats, the 3D data is described by vertices, and triangles referencing three vertices each. In addition, the XML file contains `colorgroup` tags in its resources section, that represent materials as RGBA color codes in hexadecimal format. Multiple 3D model files can be included, one of which needs to be identified in the `StartPart` relationship as the root model.

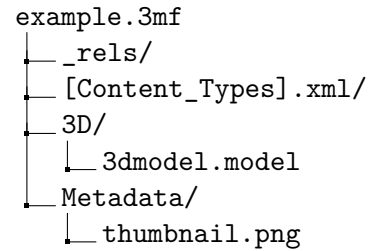


Figure 2.2.: Example structure of a 3MF file

The `object` tags each contain attributes (body id, type, color id) as well as one triangular geometry mesh representation of an object volume. The latter is represented by vertices (x-, y-, z-coordinates in the unit specified within the model’s attributes) and triangles referencing three vertices each oriented with normals pointing towards the exterior of their object.[13] Meshes of different objects may intersect (as seen in the above mentioned example’s case of wall loop modifiers) or self-intersect, in which case it must be specified which areas are inside or respectively outside the volume.

2.5. Software-specific Workflow

The overall workflow doesn’t vary greatly depending on specific software, however a few details should be noted. To explore the concept presented in this paper, *Autodesk Fusion* was used as the 3D modelling software and *OrcaSlicer* as slicer software.

2.5.1. Autodesk Fusion

Autodesk Fusion, released in 2013, offers various functionalities for computer-aided engineering (CAE) as well as product data management. The software’s CAE environment includes tools for designing, generative design, rendering, and simulation, thus supporting the entire product development process with support for a variety of digital fabrication techniques[14, p. 2].

Of particular relevance to this study are Fusion’s options for assigning *physical materials* and *appearances* to model bodies. Physical materials can be applied to entire bodies to define their thermal, mechanical, strength and other engineering properties, particularly in regards to simulations, while also determining their visual appearance. Appearances,

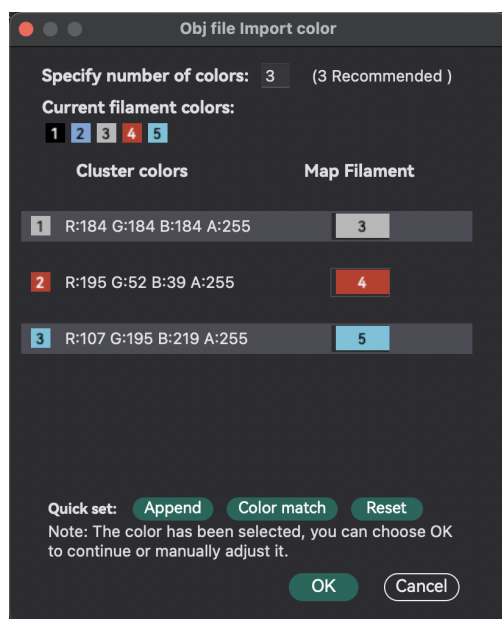
by contrast, can be assigned either to a body or individual faces, and affect only the visual rendering, overriding color and texture of previously applied materials.

Fusion projects are stored in the cloud by default, making it rare for users to directly interact with the proprietary `.f3d` file format. Generally, files are only handled locally, when exporting the model from the Fusion ecosystem. Two user interfaces provide this functionality: a general export dialogue tailored for all kinds of digital fabrication, and a 3D Print dialogue, tailored for additive manufacturing in particular. Each offers a different set of file formats corresponding to its intended purpose. The 3D Print dialogue allows exports in STL, 3MF, and OBJ formats, with the additional option to export either a single file containing all selected bodies, or separate files for each body. Furthermore, for STL and OBJ exports, the user can specify the unit type (e.g. millimetres or inches), whereas in 3MF exports this selection is predefined and therefore disabled.

2.5.2. *OrcaSlicer*

OrcaSlicer is one of the open-source slicers, originally forked from *Bambu Studio*. It inherits the core functionality of its predecessor while introducing community-driven improvements, since it is a purely community-based project with no involvement from a 3D printer manufacturer.

The workflow within *OrcaSlicer* changes subtly depending on the file format used for import. When importing a model in STL format, all bodies combined into a single file are treated as one object. If the user intends to print different parts in separate colours or apply modifiers, such as strengthening regions around a snap-fit closure, an additional step is required to split the model into separate objects within the slicer.



When importing multiple STL files at once, *OrcaSlicer* prompts the user to decide whether the files should be loaded as a single object with multiple parts or as separate objects. Loading them as a single object maintains their spatial relationships, which is useful when dealing with models that include modifiers or precisely aligned parts. In contrast, individual objects are each positioned with one surface flush to the build plate, which can lead to misalignment if the original relative positioning must be preserved.

Figure 2.3.: OBJ color mapping dialog
in *OrcaSlicer* 13

When importing an OBJ file, *OrcaSlicer* displays a dedicated dialogue asking the user how to map the file's embedded colour information to existing or new filament presets. This feature enables users to align the software's filament colours (i.e., the visual representations within *OrcaSlicer*, not necessarily corresponding to the physical filament) with the colours assigned during the modelling stage in Fusion.

Importing a 3MF file containing multiple objects, *OrcaSlicer* prompts user to decide whether to treat the file as a single object with multiple parts or as several independent objects, comparable to the behaviour when importing multiple STL or OBJ files at once. However, unlike the OBJ import, *OrcaSlicer* does not offer the user an option to map RGBA colour data embedded in the 3MF file to existing or new filament presets. Consequently, all imported parts are initially assigned the primary filament colour by default, requiring users to manually reapply materials. This limitation has been criticised by users, for example in *OrcaSlicer issue #2438*⁴.

Within *OrcaSlicer*, materials are assigned using system-defined or user-defined filament presets. The software proactively prompts users to save any modifications made to material presets, allowing for user-friendly management of customised presets tailored either to specific printers, filament types, or even individual projects. By default, when a new model or project is imported, *OrcaSlicer* automatically applies the first available user-defined filament preset.

Printing presets are handled similarly, however it has become apparent that these can only be applied globally on project level, rather than to specific individual objects within a project. This limitation becomes particularly noticeable in complex assemblies comprised of multiple components requiring different settings, as highlighted in *OrcaSlicer Issue #7769*⁵. As a result, users often resort to repetitive manual adjustments when handling multi-part models with diverse printing requirements.

⁴Single Object Multiple parts with Multiple Colours: [...] I have made a number of 3D models with multiple parts in single 3mf file. I mostly use Fusion 360 for models and during modelling I assign colours to each part. But when I upload this 3mf file to the slicer, It assigns the primary filament colour to all the parts. I was wondering if the slicer would be able to distinguish the parts with different colours and assign either the original colour to the parts or even random colours(which can be changed later). For example if my 3mf file has 5 colour parts, the slicer should be to detect them. Then it should find which colour has the maximum spread by area and assign the filament 1 to it. The second largest should be assigned the second filament and so on till the last.[...]
<https://github.com/SoftFever/OrcaSlicer/discussions/2438>

⁵Copy/paste per-object settings: When planning/building large projects in-slicer it often becomes an extremely tedious task to click all the objects you want a specific set of settings to apply to. The larger the assembly/project you are planning, the easier it is to miss a body and make a mistake. If you "assemble" models together you cannot edit multiple sub-objects across assemblies.
<https://github.com/SoftFever/OrcaSlicer/issues/7769>

Chapter 3.

Concept

In practical additive manufacturing workflows, especially within iterative design processes, maintaining consistency and traceability of printing parameters across multiple model revisions presents a recurring challenge. As mentioned before, designers often need to adjust filament and printing parameters as well as modifiers between print iterations. However, these configurations are typically defined and stored within the slicer software rather than the modelling environment. As a result, when models are modified and re-exported, relevant printing information is easily lost or must be manually reconstructed.

This separation between modelling and slicing environments can lead to inefficiencies, particularly in projects that require several iterations, complex geometries, or multiple materials. Without a systematic method for documenting and reusing printing parameters directly within the model, users are forced to rely on external notes or memory, which increases the likelihood of inconsistency and error.

3.1. Design Requirements

Observing this problem motivated the project to improve the integration between 3D modelling and slicing workflows by enabling the storage and transfer of essential printing information alongside the model itself as print hints. The underlying goal is to enhance usability, reproducibility, and workflow efficiency without introducing additional software complexity or disrupting established design practices. Thus the following requirements were identified.

1. The solution should integrate seamlessly into the existing modelling workflow without requiring additional software installations or complex documentation procedures.
2. It maintains user flexibility to refine parameters iteratively, acknowledging that printing quality depends both on the model geometry and the chosen print settings.

3. It supports portability and long-term accessibility of the parameter data using existing file standards.

3.2. Concept Development

The initial concept involved creating a custom add-in for *Fusion* that would allow users to manually attach print hints to parts of the model. However, this approach contradicted the primary design goals of simplicity and minimal workflow disruption. The requirement for additional software installation and maintenance represented an unnecessary barrier for users.

To overcome these limitations, the idea evolved toward adopting native functionality within *Fusion* — specifically, the use of appearances. Appearances in *Fusion* are visual attributes that define colour, opacity, and reflectivity without influencing the physical material properties or simulation behaviour. In addition to being fully integrated into *Fusion*’s native environment, appearances are also automatically stored within the project file, and easily managed through user-defined libraries that can be stored across separate projects.

By repurposing appearances as carriers of print hint information, users could assign distinct visual markers to model components, corresponding to specific printing or filament presets. This method meets the objectives of low technical overhead and ease of use, while at the same time utilising existing mechanisms for data persistence and portability.

3.3. File Format Considerations

In order to meet the aforementioned requirements, it is necessary to export the print hints together with the model. The STL format was disregarded on account of the limited information it encodes, as outlined in 2.4.1. Thus, OBJ and 3MF were considered as potential candidates, since both have the ability to represent material data.

Following a detailed evaluation, 3MF was determined to be the more suitable option for this project. A key factor is its ability to natively embed material information within a single, self-contained file structure, in contrast to OBJ, which utilises a separate material library file. Consequently, this complicates data management and increases the risk of information loss or inconsistencies during file transfer. Moreover, 3MF’s open and extensible architecture allows for future integration of additional print hints. Its human-readable XML-based format further supports transparency and verifiability throughout

the development process, facilitating both debugging and documentation. Collectively, these characteristics make 3MF the more robust and forward-compatible choice for representing models enriched with print-related metadata.

3.4. User Interaction

To complement the embedding of print hints in the model, an import dialog was envisioned for the slicer application. The design was inspired by existing implementations such as *OrcaSlicer*'s handling of OBJ imports, where users can map model colours to filament presets.

In the proposed approach, this dialog would be extended to allow:

- Mapping of model appearances to both filament and printing presets,
- Immediate designation of bodies as modifiers, and
- Creation or selection of presets directly from the interface.

Upon initial use, the user could select default or newly created presets. When the same appearance (colour) is encountered in future imports, the system would reference a configuration file to suggest previously associated presets automatically. This mechanism reduces repetitive configuration tasks while preserving user control and flexibility for iterative refinement.

Chapter 4.

Technical Implementation

This chapter outlines the technical implementation of the proposed concept, with a particular focus on the analysis and modification of the 3MF file structure and its integration into the *OrcaSlicer* environment. The goal was to enable the transfer of model appearances from Fusion to OrcaSlicer in a manner that preserves and utilises colour-based metadata as print hints. To demonstrate the feasibility of the concept, the implementation focused on enabling the application of presets to components based on their associated colours during import. For this purpose, *OrcaSlicer*'s code repository was forked, using Git as a version control system. The subsequent sections illustrate the process of analysing *Fusion*'s 3MF export behaviour, identifying relevant colour data structures, adapting *OrcaSlicer*'s import logic, and extending the user interface to support the proposed functionality.

4.1. Technical Analysis of the 3MF File Structure

Before the implementation, a comprehensive analysis of the 3MF file format was conducted to determine how *Fusion* stores appearance data. The 3MF format organises the main model-related data in hierarchical XML structures.

Per the additional Materials and Properties Extension specification[15], colour information can be encoded in several different ways. One such option is the colour group, which *Fusion* utilises. Each group is identified by a unique parameter ID, referenced per object as `pid`, and contains one or more `<color>` entries. The colours are defined using an RGBA colour code in hexadecimal format and are implicitly assigned a 0-based index. In compliance with the specification, the alpha channel is encoded but not utilised, and can therefore be disregarded in this context. In the file structure example A.3, these sections can be found in lines 4-40 (`<resources>`), enclosing the `<m:colorgroup>` and `<object>` tags.

4.1.1. *Fusion*'s Export Behaviour

An initial uncertainty concerned whether *Fusion* exported appearance data by default, as no colours appeared during import into *OrcaSlicer*. Upon closer inspection, it became evident that *Fusion* does, in fact, include appearance information; however, each appearance is represented as a colour group containing two colour entries. The first entry typically approximates the visual appearance assigned in *Fusion*, while the second entry is consistently a light grey tone.

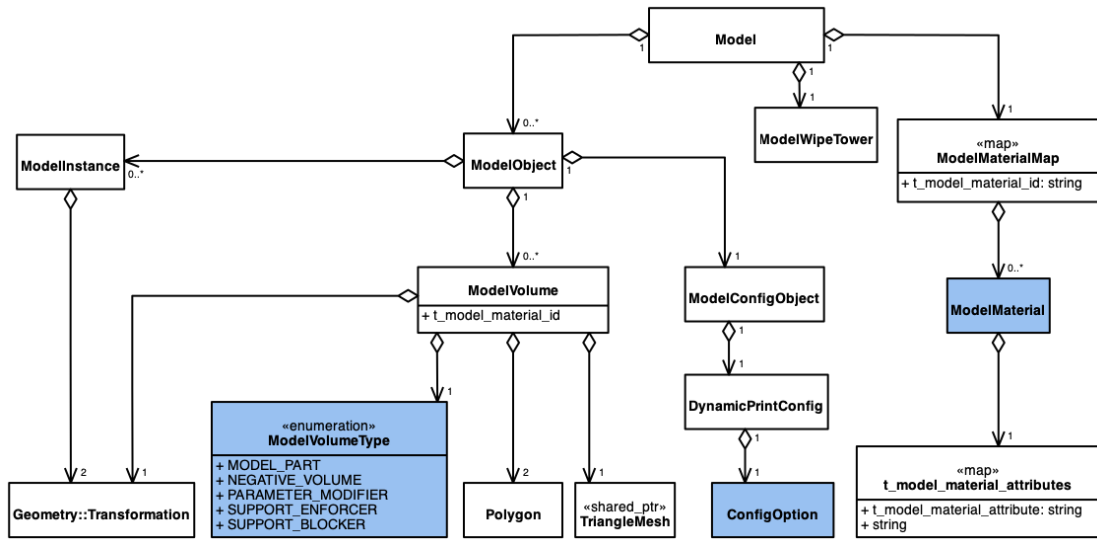
Assigning multiple appearances to a single component merely resulted in additional `<color>` tags within the same group, without altering the pattern that the final colour remained grey. Since *Fusion* allows appearances and physical materials to be applied independently, it was initially hypothesised that one colour might represent the appearance while the other corresponded to the physical material. However, neither documentation nor experimental evidence substantiates this assumption, and the precise purpose of the second colour remains undetermined.

4.2. Analysis *OrcaSlicer*'s 3MF Import Logic

In order to understand why *Fusion*'s exported colours were not displayed, the source code of *OrcaSlicer* was examined thoroughly. Preliminary analysis indicated that the function responsible for parsing XML colour data was not being executed. However, upon closer inspection it became apparent that the function was indeed called and populated data structures — albeit not the ones represented in the `Model` class (4.2).

The observed behaviour could be traced to the handling of colour groups during import. The colours within each group were processed sequentially, but only the last colour in the group was retained in memory, effectively overwriting the others. This behaviour explains why only a single (typically grey) colour appeared after import.

To address this, the import logic was modified to preserve the *first* colour in each group instead of the *last*. Following this adjustment, all colours defined in the 3MF file were correctly interpreted and handled within *OrcaSlicer*. This modification enabled the application to assign one filament preset to each colour detected across multiple model components. Specifically, it applied the user's saved filament presets in the saved order consecutively to each color it encountered.

Figure 4.1.: Excerpt of *OrcaSlicer*'s Model class with Relevant Attributes highlighted

4.2.1. Internal Data Handling

After confirming that colours could be successfully imported, the next step was to identify the location of this information within *OrcaSlicer*'s internal model representation. Contrary to initial assumptions, the colour attributes were not associated with the material attributes of individual model instances (**ModelMaterialMap**).

Instead, the data was found to be associated with the extruder configurations, which handle the assignment of filaments and their corresponding parameters. In *OrcaSlicer*, extruder information is implemented as a general **ConfigOption** object (see 4.2), a wrapper class encapsulating a key-value pair with support for multiple variable types. Since this structure abstracts the data type without explicitly defining its contents, identifying and tracing the relevant information introduced additional complexity to the analysis of the underlying codebase.

4.3. User Interface Adaption

The user interface design for importing 3MF models with colour mappings was inspired by the OBJ import workflow, which allows users to map model colours to existing filament presets, as shown in 2.5.2. Therefore, the corresponding UI logic and data flow were analysed to understand how the system collects, displays, and processes colour data.

//**TODO** Insert short explanation of the relevant functions, e.g., where and how OBJ import populates the UI.

Because 3MF and OBJ formats are handled through different pipelines, several adjustments were required. The imported colours were collected and converted into a vector<RGBA> structure. Since 3MF represents colours in hexadecimal form, a conversion to a floating-point representation was necessary to match the expected input format of the UI dialog (four decimal values corresponding to RGBA). Existing code segments for colour conversion within the *OrcaSlicer* repository were reused to ensure consistency.

The dialog also required additional information about the current extruder configuration, including the colour of each extruder, the ID of the primary extruder, and associated filament identifiers. Once provided, the dialog correctly displayed the imported colours and allowed manual mapping to filament presets.

Although additional analysis and implementation steps had been intended, unforeseen technical challenges and limited time prevented their completion within the scope of the project.

Chapter 5.

Results

The implementation process primarily focused on validating the conceptual feasibility of integrating print hints into the 3D modelling and slicing workflow. While substantial progress was made in analysing the 3MF structure and adapting OrcaSlicer’s import mechanism, the practical implementation was limited in scope due to the complexity of the existing code base and time constraints.

The project successfully demonstrated that appearance-based print metadata can be extracted from Fusion’s 3MF exports, interpreted, and displayed in OrcaSlicer. The implemented modifications to the import logic confirmed that colour information, once properly parsed, can serve as a viable medium for associating model components with specific filament presets or other printing parameters.

Furthermore, the exploratory work yielded a clearer understanding of how Fusion’s colour representation functions internally and how OrcaSlicer manages its data structures during model import. These insights establish an important technical foundation for extending slicer software to interpret embedded model metadata more effectively.

Despite the conceptual success, several limitations remain. Most notably, the precise rationale behind OrcaSlicer’s original behaviour — saving the last rather than the first colour per group — requires further investigation, as altering this mechanism might inadvertently affect other functionalities dependent on that logic.

Additionally, aspects such as test coverage, code modularity, and localisation were not comprehensively addressed, primarily due to the exploratory nature of this work. A systematic testing strategy and internationalisation support would be essential prerequisites before any potential integration into the main code base could be proposed.

5.1. Planned Work

Building upon the achieved results, the next steps for development would include:

- Comprehensive verification of the modified 3MF import process to ensure compatibility with all supported features and file variations.
- Implementation of persistent preset management, allowing users to automatically reapply previously associated print or filament presets during subsequent imports.
- Extension of the user interface, enabling the creation, selection, and modification of presets directly within the import dialog.
- Definition of a configuration schema for storing colour–preset associations to facilitate reproducibility and long-term usability.

In a more advanced development stage, the project could be presented to the *OrcaSlicer* developer community for peer review and potential integration into the main repository. Such collaboration would also ensure adherence to the project’s coding conventions and user experience guidelines.

Chapter 6.

Conclusion

This work addressed the challenge of repetitive manual configuration during 3MF file import in slicer software. Through conceptual exploration and partial implementation, it demonstrated that metadata embedded in a 3D model — specifically, appearance information — can be leveraged to streamline the import process and improve parameter consistency.

The research and experimentation presented throughout Chapters 3 and 4 confirm the technical feasibility of this approach, even though further development is required for full integration. The 3MF format, with its open, extensible, and human-readable structure, proved particularly well-suited to this goal. Its capabilities remain underutilised in current additive manufacturing workflows, representing an opportunity for innovation in user-oriented print preparation tools.

While the implemented prototype represents only a preliminary stage, it establishes a foundation for a more user-friendly and efficient workflow between modelling and slicing environments. A wider adoption of the 3MF format’s capabilities could foster a more standardised exchange of print-related data and encourage community-driven enhancement of open-source slicer tools.

The proposed print hints concept demonstrates a practical method for integrating usability improvements directly into existing workflows. However, the ongoing evolution of slicer technologies and the diversity of user requirements leave ample room for alternative or complementary approaches. Future contributions from the community could further refine or expand upon the ideas presented in this work, ultimately contributing to a more seamless and accessible additive manufacturing process.

Appendix A.

Supplemental Information

```
1 solid ASCII
2   facet normal 1.000000e+00 0.000000e+00 -0.000000e+00
3     outer loop
4       vertex 9.900000e+01 -3.480000e+01 3.250000e+01
5       vertex 9.900000e+01 -3.480000e+01 3.500000e+01
6       vertex 9.900000e+01 -4.020000e+01 3.250000e+01
7     endloop
8   endfacet
9   facet normal 1.000000e+00 0.000000e+00 0.000000e+00
10    outer loop
11      vertex 9.900000e+01 -4.020000e+01 3.250000e+01
12      vertex 9.900000e+01 -3.480000e+01 3.500000e+01
13      vertex 9.900000e+01 -4.020000e+01 3.500000e+01
14    endloop
15  endfacet
16  ...
17 endsolid
```

Listing A.1: Abbreviated Example of STL's 3D Model Representation

```

1 # WaveFront *.obj file (generated by Autodesk ATF)
2
3 mtl lib Pandoras_Box.mtl
4
5 g Body1
6
7 v 99.000000 -34.800000 32.500000
8 v 99.000000 -34.800000 35.000000
9 ...
10 vt 0.000000 0.000000 0.000000
11 vt 0.250000 0.000000 0.000000
12 ...
13 vn 0.100000 0.000000 0.000000
14 vn 0.100000 0.000000 0.000000
15
16 usemtl Nylon_12_(with_Formlabs_Fuse_1_3D_Printer)
17 f 1/1/1 2/2/2 4/3/3
18 f 4/3/3 2/2/2 3/4/4
19 ...
20
21 g Body2
22
23 v 99.000000 -35.000000 35.000000
24 v 99.000000 -35.000000 32.500000
25 ...
26 vt -0.250000 0.000000 0.000000
27 vt -0.000000 0.000000 0.000000
28 ...
29 vn -0.100000 0.000000 0.000000
30 vn -0.100000 0.000000 0.000000
31 ...
32 usemtl Nylon_12_(with_Formlabs_Fuse_1_3D_Printer)_(1)
33 f 369/833/809 370/834/810 372/835/811
34 f 372/835/811 370/834/810 371/836/812
35 ...
36
37 # 3464 vertices
38 # 7296 texture params
39 # 7036 normals

```

```
40 # 6904 facets
41
42 # 11 groups
43
44 # WaveFront *.mtl file (generated by Autodesk ATF)
45
46 newmtl Nylon_12_(with_Formlabs_Fuse_1_3D_Printer)
47 Kd 0.705882 0.701961 0.701961
48
49 newmtl Nylon_12_(with_Formlabs_Fuse_1_3D_Printer)_(1)
50 Kd 0.941176 0.968627 0.972549
```

Listing A.2: Abbreviated Example of Wavefront OBJ's Representation of a 3D Model and Material Collection

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <model xmlns="http://schemas.microsoft.com/3dmanufacturing/core
  /2015/02" unit="millimeter" xml:lang="en-US" ...>
3   <metadata name="Title" type="QName" preserve="1">Pandoras
     Box</metadata>
4   <resources>
5     <m:colorgroup id="2">
6       <m:color color="#B4B3B3FF" />
7       <m:color color="#A0A0A0FF" />
8     </m:colorgroup>
9     <m:colorgroup id="4">
10      <m:color color="#449648FF" />
11      <m:color color="#A0A0A0FF" />
12    </m:colorgroup>
13    ...
14    <object id="1" name="Body1" type="model" p:UUID="88b92134
      -b51b-4129-a501-3cb7a6b35a19" pid="2" pindex="0">
15      <mesh>
16        <vertices>
17          <vertex x="99.000000" y="-34.800000" z="
            32.500000" />
18          <vertex x="99.000000" y="-34.800000" z="
            35.000000" />
19          ...
20        </vertices>
21        <triangles>
22          <triangle v1="0" v2="1" v3="3" />
23          <triangle v1="3" v2="1" v3="2" />
24          ...
25        </triangles>
26      </mesh>
27    </object>
28    <object id="3" name="Body10" type="model" p:UUID="83
      b1ffc1-fe3a-41ff-b2d8-709e401b58c0" pid="4" pindex="0"
      >
29    <mesh>
30      <vertices>
31        <vertex x="97.000000" y="44.000000" z="26.000000
          " />

```



```
32         ...
33     </vertices>
34     <triangles>
35         <triangle v1="0" v2="1" v3="3" />
36         ...
37     </triangles>
38 </mesh>
39 </object>
40 </resources>
41 <build p:UUID="9062348e-a030-4f92-86b2-786a03cd804d">
42     <item objectid="1" p:UUID="df0ef4a8-b05c-41c3-a041-
43         f6ca9d41a68c" />
44     <item objectid="3" p:UUID="2dd63bd5-069d-405e-9b05-0
45         fd5f5a71f72" />
46     ...
47 </build>
48 </model>
```

Listing A.3: Abbreviated Example of 3MF's 3dmodel.model File

List of Figures

1.1. 3D model of snap-fit closure box as example. A: Top. B: Bottom. C: Integrated text to be printed in different colour. D: Closure with surrounding modifier.	2
2.1. Iterative workflow in 3D printing highlighting the step to be optimised	5
2.2. Example structure of a 3MF file	12
2.3. OBJ color mapping dialog in <i>OrcaSlicer</i>	13
4.1. Excerpt of <i>OrcaSlicer</i> 's Model class with Relevant Attributes highlighted . . .	20

List of Listings

A.1. Abbreviated Example of STL's 3D Model Representation	25
A.2. Abbreviated Example of Wavefront OBJ's Representation of a 3D Model and Material Collection	26
A.3. Abbreviated Example of 3MF's 3dmodel.model File	27

Bibliography

- [1] J. A. Jacko, *Human Computer Interaction Handbook: Fundamentals, Evolving Technologies, and Emerging Applications, Third Edition*. Milton, UNITED KINGDOM: Taylor & Francis Group, 2012.
- [2] Z. Yan, M. S. Dhaygude, and H. Peng, “Make Making Sustainable: Exploring Sustainability Practices, Challenges, and Opportunities in Making Activities,” in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’25. New York, NY, USA: Association for Computing Machinery, Apr. 2025, pp. 1–14. doi:[10.1145/3706598.3713665](https://doi.org/10.1145/3706598.3713665)
- [3] S. Landwehr Sydow, M. Jonsson, and J. Tholander, “Machine Sensibility: Unpacking the Embodied and Situated Dimensions of 3D Printing,” in *Proceedings of the 11th Nordic Conference on Human-Computer Interaction: Shaping Experiences, Shaping Society*. Tallinn Estonia: ACM, Oct. 2020, pp. 1–13. doi:[10.1145/3419249.3420166](https://doi.org/10.1145/3419249.3420166)
- [4] N. Hudson, C. Alcock, and P. K. Chilana, “Understanding Newcomers to 3D Printing: Motivations, Workflows, and Barriers of Casual Makers,” in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 384–396. doi:[10.1145/2858036.2858266](https://doi.org/10.1145/2858036.2858266)
- [5] J. Walter-Herrmann, *FabLab: Of Machines, Makers and Inventors*, 1st ed., ser. Kultur- und Medientheorie. Bielefeld: transcript, 2014.
- [6] J. Horvath and R. Cameron, *Mastering 3D Printing in the classroom, library, and lab*. New York: Apress, 2018.
- [7] B. Subbaraman, N. Bursch, and N. Peek, “It’s Not the Shape, It’s the Settings: Tools for Exploring, Documenting, and Sharing Physical Fabrication Parameters in 3D Printing,” in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’25. New York, NY, USA: Association for Computing Machinery, Apr. 2025, pp. 1–19. doi:[10.1145/3706598.3713354](https://doi.org/10.1145/3706598.3713354)

- [8] F. Jasche, P. Weber, S. Liu, and T. Ludwig, “PrintAssist—a conversational human-machine interface for 3D printers,” *i-com*, vol. 22, no. 1, pp. 3–17, Apr. 2023, publisher: Oldenbourg Wissenschaftsverlag. doi:[10.1515/icom-2022-0045](https://doi.org/10.1515/icom-2022-0045)
- [9] E. Buehler, S. K. Kane, and A. Hurst, “ABC and 3D: opportunities and obstacles to 3D printing in special education environments,” in *Proceedings of the 16th international ACM SIGACCESS conference on Computers & accessibility - ASSETS '14*. Rochester, New York, USA: ACM Press, 2014, pp. 107–114. doi:[10.1145/2661334.2661365](https://doi.org/10.1145/2661334.2661365)
- [10] I. Gibson, D. W. Rosen, and B. Stucker, *Additive manufacturing technologies: rapid prototyping to direct digital manufacturing*. New York London: Springer, 2010.
- [11] A. Kumar, P. Kumar, R. K. Mittal, and H. Singh, “Printing file formats for additive manufacturing technologies,” in *Advances in Additive Manufacturing Artificial Intelligence, Nature-Inspired, and Biomanufacturing*. Elsevier, 2023, pp. 87–102. doi:[10.1016/B978-0-323-91834-3.00006-5](https://doi.org/10.1016/B978-0-323-91834-3.00006-5)
- [12] J. Rossel, V. Mladenov, and J. Somorovsky, “Security Analysis of the 3MF Data Format,” in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*. Hong Kong China: ACM, Oct. 2023, pp. 179–194. doi:[10.1145/3607199.3607216](https://doi.org/10.1145/3607199.3607216)
- [13] 3MF Consortium, “3MF Core Specification & Reference Guide,” International Organization for Standardization, Standard, Feb. 2025. Available: <https://3mf.io/spec/core-v1-3-0/> (Accessed 2025-06-21).
- [14] S. Junk, *Autodesk® Fusion - kurz und bündig: Einstieg in 3D-Druck und CNC-Biegen*, 4th ed. Wiesbaden: Springer Vieweg, 2025.
- [15] 3MF Consortium, “Materials Extension,” International Organization for Standardization, Standard, Feb. 2025. Available: <https://3mf.io/spec/materials-v1-2-1/> (Accessed 2025-09-10).
- [16] T. Ludwig, A. Boden, and V. Pipek, “3D Printers as Sociable Technologies: Taking Appropriation Infrastructures to the Internet of Things,” *ACM Trans. Comput.-Hum. Interact.*, vol. 24, no. 2, pp. 17:1–17:28, Apr. 2017. doi:[10.1145/3007205](https://doi.org/10.1145/3007205)
- [17] T. Kosch, J. Karolus, J. Zagermann, H. Reiterer, A. Schmidt, and P. W. Woźniak, “A Survey on Measuring Cognitive Workload in Human-Computer Interaction,” *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–39, Dec. 2023. doi:[10.1145/3582272](https://doi.org/10.1145/3582272)